MASTER YOUR LOGIC BUILDING

(COMPLETE C PREREQUISITES BEFORE STARTING DSA)

Original 6 phases (logic building) + 4 new phases (C-specific: functions, pointers, structures, dynamic memory) required before DSA in C

Why the Original Sheet Isn't Enough for C

Phases 1–6 (Conditionals, Loops, Recursion, Arrays, Strings, Mixed Logic) build pure algorithmic thinking and work for any language. But DSA in C is built on top of four things those phases never touch:

1. Functions — DSA problems are almost always solved as functions (e.g., int search(int arr[], int n, int key)), with parameters, return values, and scope. Without this, you can't even write a clean linked-list insert function.
2. Pointers — in C, a linked list node, a tree node, or a graph adjacency list literally cannot exist without pointers. This is the single biggest gap.
3. Structures (struct) — every DSA data structure (Node, Stack, Queue, Graph) is defined as a struct in C. Without this, there is nothing to "link" or "point" to.
4. Dynamic memory allocation (malloc/calloc/realloc/free) — arrays in Phase 4 are fixed-size. Real DSA needs memory created and destroyed at runtime.

Recommended order: do Phases 1–6 (below) first for pure logic, then Phases 7–10 before touching DSA topics like Linked Lists, Stacks, Queues, Trees, or Graphs. Skipping 7–10 is the #1 reason students "understand DSA in theory but can't code it in C."

Phases 1–6 — Original Logic Building (as provided)

Phase 1 — Conditional Thinking (If–Else, Boolean Logic)

Goal: Understand how to make decisions using conditions.

Topics covered: Relational operators, logical operators, nested if, multiple conditions.

Target Questions: 50

Level 1: Simple Conditions (Getting started)

1. Take a number and print whether it's positive, negative, or zero.
2. Check if a number is even or odd.
3. Check if a number is divisible by 5.
4. Check if a number is divisible by both 3 and 5.
5. Check if a given year is a leap year.
6. Take two numbers and print the larger one.
7. Take three numbers and print the largest.
8. Take a temperature value and print "Cold", "Warm", or "Hot" using range conditions.
9. Take a character and check if it's a vowel or consonant.
10. Take a character and check whether it's uppercase, lowercase, a digit, or a special character.

Level 2: Nested If & Multiple Conditions

1. Take three sides and check if they form a valid triangle.
2. If the sides form a valid triangle, determine whether it is equilateral, isosceles, or scalene.
3. Take marks (0–100) and print the corresponding grade (A/B/C/D/F).
4. Check if one of two given numbers is a multiple of the other.
5. Take the hour of the day (0–23) and print "Good Morning", "Good Afternoon", "Good Evening", or "Good Night".
6. Check voting eligibility for a given age (18+).
7. Take two numbers and determine whether both are even, both are odd, or one is even and one is odd.
8. Take an alphabet character and check if it lies between 'a' and 'm' or 'n' and 'z'.
9. Take a day number (1–7) and print the corresponding day name.
10. Take a month number (1–12) and print the number of days in that month (ignore leap years).

Level 3: Math and Number Logic

1. Take a 3-digit number and check if all digits are distinct.
2. Take a 3-digit number and determine if the middle digit is the largest, smallest, or neither.
3. Take a 4-digit number and check if the first and last digits are equal.
4. Check whether a given integer is single-digit, double-digit, or multi-digit.
5. Check if a number is a multiple of 7 or ends with 7.
6. Take coordinates (x, y) and determine which quadrant the point lies in.
7. Check if an amount can be evenly divided into 2000, 500, and 100 currency notes.
8. Check if a number lies within the range [100, 999].
9. Take two angles of a triangle and compute the third angle.
10. Check whether a number is a perfect square (without using the square root function).

Level 4: Logical Operators & Compound Statements

1. Take a character and check if it is a letter, a digit, or neither.
2. Take a number and print "Fizz" if divisible by 3, "Buzz" if divisible by 5, and "FizzBuzz" if divisible by both.
3. Take three numbers and print the median value (neither maximum nor minimum).
4. Take 24-hour time (hours and minutes) and print whether it is AM or PM.
5. Take income and age, and check if eligible for tax (age > 18 and income > 5 L).
6. Take two numbers and check if both are positive and their sum is less than 100.
7. Take a single digit (0–9) and print its word form ("Zero" to "Nine").
8. Take a weekday number (1–7) and determine if it is a weekday or weekend.
9. Take electricity units consumed and calculate the bill as per slabs (using if-else).
10. Take a password string and check basic rules (length ≥ 8 and contains at least one digit).

Level 5: Creative / Tricky Logical Scenarios

1. Take coordinates (x, y) and check if the point lies on the X-axis, Y-axis, or at the origin.
2. Take three numbers and check if they can form a Pythagorean triplet.
3. Take day and month and check if it forms a valid calendar date (ignoring leap years).
4. Take time (hours and minutes) and print the smaller angle between the hour and minute hands.
5. Take three numbers and check if they are in arithmetic progression.
6. Take three numbers and check if they are in geometric progression.
7. Take a 3-digit number and check if the sum of the first and last digit equals the middle digit.
8. Take an integer (1–9999) and check if the sum of its digits is greater than the product of its digits.
9. Take two dates (day and month) and determine which one comes first in the calendar.
10. Take a year and print the corresponding century (e.g., "19th century", "20th century").

Phase 2 — Looping & Patterns (Iteration & Flow)

Goal: Master loops, iteration, and dry-run thinking.

Topics covered: for, while, nested loops, break/continue, mathematical series.

Target Questions: 40–50

Level 1: Basic Looping (Introductory)

1. Print numbers from 1 to 10.
2. Print all even numbers between 1 and 100.
3. Print all odd numbers between 1 and 100.
4. Print numbers from 10 down to 1.
5. Print the table of a given number (n × 1 to n × 10).
6. Print the sum of first n natural numbers.
7. Print the sum of all even numbers up to n.
8. Print the sum of all odd numbers up to n.
9. Print the factorial of a given number.
10. Print the product of digits of a given number.

Level 2: Number-based Looping Logic

1. Count the number of digits in a given number.
2. Print the reverse of a given number.
3. Check if a number is a palindrome.
4. Find the sum of digits of a number.
5. Check if a number is an Armstrong number.
6. Check if a number is a perfect number.
7. Print all prime numbers between 1 and 100.
8. Check if a number is prime or not.
9. Print Fibonacci series up to n terms.
10. Print sum of first n terms of Fibonacci series.

Level 3: Mathematical & Logical Patterns

1. Print the squares of numbers from 1 to n.
2. Print cubes of numbers from 1 to n.
3. Print all numbers between a and b divisible by 7.
4. Find HCF (GCD) of two numbers using loops.
5. Find LCM of two numbers using loops.
6. Print all factors of a given number.
7. Find the sum of all factors of a number.
8. Check if a number is a strong number (sum of factorials of digits = number).
9. Print first n terms of an arithmetic progression (a, d).
10. Print first n terms of a geometric progression (a, r).

Level 4: Pattern Printing (Stars & Numbers)

1. Nested-loop pattern printing (star/number triangles, pyramids, diamonds) — practice on the referenced "Strong Your Logic Building" sheet; must-do for nested-loop fluency.

Level 5: Logical Loop Combinations

1. Print all numbers whose sum of digits is even (1–100).
2. Count how many numbers between 1–500 are divisible by 7 but not by 5.
3. Print all numbers that are palindromes between 1–500.
4. Print numbers between 1–100 whose digits add up to a multiple of 3.
5. Find the smallest and largest digit in a given number.
6. Print all numbers from 1–n whose binary representation has an even number of 1s.
7. Print a pattern where each row i prints i*i.
8. Print factorial of each number from 1 to n.
9. Print the sum of all odd digits and even digits separately in a number.
10. Take 5 numbers as input. If the user enters 0, skip it using continue. At the end, print the sum of all non-zero numbers entered.

Phase 3 — Recursion (Thinking in Self-Reference)

Goal: Develop logical decomposition and base-condition thinking.

Topics covered: recursive definition, base cases, call stack tracing.

Target Questions: 30–40

Level 1: Foundation of Recursion (Base + Recursive Case)

1. Print numbers from 1 to n using recursion.
2. Print numbers from n down to 1 using recursion.
3. Print only even numbers from 1 to n recursively.
4. Print only odd numbers from 1 to n recursively.
5. Print sum of first n natural numbers recursively.
6. Print factorial of a number recursively.
7. Calculate power of a number (x^n) using recursion.
8. Find nth Fibonacci number recursively.
9. Print Fibonacci series up to n terms recursively.
10. Find sum of digits of a number recursively.

Level 2: Number-based Recursive Thinking

1. Count the number of digits in a number recursively.
2. Reverse a number recursively.
3. Check if a number is a palindrome using recursion.
4. Find product of digits of a number recursively.
5. Find GCD (HCF) of two numbers using Euclid's algorithm recursively.
6. Convert a number to binary recursively.
7. Print digits of a number in words recursively (e.g., 123 → "one two three").
8. Calculate the sum of first n even numbers recursively.
9. Calculate the sum of first n odd numbers recursively.
10. Find nCr (Combination formula) recursively using Pascal's relation.

Level 3: Pattern & Printing Problems

1. Print a line of n stars recursively.
2. Print a square of stars recursively (n×n).
3. Print a triangle of stars recursively (top-down).
4. Print a triangle of stars recursively (bottom-up).

5. Print pattern of numbers recursively (1 to n each row).
6. Print reverse triangle pattern recursively.
7. Print multiplication table of n recursively.
8. Print numbers in increasing and decreasing order in same function.
9. Print sum of series 1 + 2 + 3 + ... + n recursively and display each step.
10. Print pattern of characters (A, AB, ABC, ...) recursively.

Level 4: String-based Recursion

1. Reverse a string using recursion.
2. Check if a string is palindrome using recursion.
3. Count vowels in a string recursively.
4. Remove all spaces from a string recursively.
5. Replace all occurrences of a character (say 'a' → 'x') recursively.
6. Remove all occurrences of a character from a string recursively.
7. Print all characters of a string one by one recursively.
8. Print the string in reverse order recursively (without using loops).
9. Convert a string to uppercase recursively.
10. Count consonants and vowels separately using recursion.

Phase 4 — Basic Arrays (Iterative Logical Thinking)

Goal: Build the ability to handle a collection logically.

Topics covered: traversal, frequency, simple manipulation, aggregations.

Target Questions: 30–40

Level 1: Fundamentals of Arrays

1. Input n and take n integers into an array; print them.
2. Find the sum of all elements in an array.
3. Find the average of array elements.
4. Find the maximum element in an array.
5. Find the minimum element in an array.
6. Count how many elements are positive, negative, or zero.
7. Count how many elements are even and odd.
8. Find the index of the maximum element.
9. Find the index of the minimum element.
10. Take n elements and print only those greater than a given value k.

Level 2: Searching & Counting Logic

1. Input an element x — check if it exists in the array.
2. Count how many times a given element appears.
3. Find the first occurrence of a given number.
4. Find the last occurrence of a given number.
5. Check if all elements in an array are unique.
6. Find the sum of even elements only.
7. Find the sum of odd elements only.
8. Find the count of prime numbers in the array.

9. Count how many numbers are divisible by 3 and 5 both.
10. Count how many elements are perfect squares.

Level 3: Transformation & Manipulation

1. Create a new array containing squares of all numbers.
2. Create a new array containing only even elements.
3. Replace every negative number with 0.
4. Replace all even numbers with 1 and all odd with 0.
5. Swap the first and last elements of the array.
6. Reverse an array (without using built-in reverse).
7. Rotate an array by one position to the left.
8. Rotate an array by one position to the right.
9. Swap alternate elements (1st ↔ 2nd, 3rd ↔ 4th, etc.).
10. Copy one array to another manually.

Level 4: Aggregate & Comparative Thinking

1. Compare two arrays — check if they are equal (same elements & order).
2. Compare two arrays — check if they contain the same elements (ignore order).
3. Merge two arrays into a third array.
4. Find the common elements between two arrays.
5. Find elements that are in one array but not in the other.
6. Count how many elements are common between two arrays.
7. Find element-wise sum of two arrays (A[i] + B[i]).
8. Find element-wise product of two arrays.
9. Create a frequency array of numbers (count occurrence of each number).
10. Print all elements that appear more than once.

Level 5: Logical & Applied Array Problems

1. Check if the array is sorted in ascending order.
2. Check if the array is sorted in descending order.
3. Find the second largest element in an array.
4. Find the second smallest element in an array.
5. Find the difference between the largest and smallest element.
6. Find the sum of all elements except the largest and smallest.
7. Count how many pairs of elements have a sum equal to a given number k.
8. Count how many elements are greater than the average of the array.
9. Print the frequency of each distinct element.
10. Print all unique elements (those that occur exactly once).

Phase 5 — Strings (Basic Logic Building)

Target Questions: 50

Category 1: Basic String Handling

1. Take a string input and print its length.
2. Print the first and last character of a string.

3. Convert all characters of a string to uppercase.
4. Convert all characters of a string to lowercase.
5. Count how many characters (excluding spaces) are in the string.
6. Count how many words are in a sentence.
7. Take two strings and print them concatenated.
8. Compare two strings lexicographically (like dictionary order).
9. Print the ASCII value of each character in a string.
10. Check whether the string is empty or not.

Category 2: Counting & Character Analysis

1. Count how many vowels and consonants are in a string.
2. Count the number of digits, letters, and special characters in a string.
3. Count how many uppercase and lowercase letters a string has.
4. Find the frequency of each character in a string (without using a map).
5. Count how many spaces are there in a sentence.
6. Count how many times a given character appears in a string.
7. Count how many alphabets are before 'm' and after 'm' in a given string.
8. Count how many substrings start and end with the same character (simple logic).
9. Print how many words start with a vowel in a sentence.
10. Count how many words end with 's'.

Category 3: Reversing & Palindromic Thinking

1. Reverse a string without using built-in reverse.
2. Reverse each word in a sentence.
3. Reverse the order of words in a sentence.
4. Check whether a string is a palindrome.
5. Check if two strings are the reverse of each other.
6. Print the middle character(s) of a string.
7. Print the second half of the string in reverse.
8. Remove the first and last character and print the remaining string.
9. Reverse only characters, keeping digits in place.
10. Reverse string but skip spaces.

Category 4: Character & Word Manipulation

1. Remove all vowels from a string.
2. Remove all spaces from a string.
3. Replace all vowels with '*'.
4. Replace all spaces with '_'.
5. Print the string after removing all digits.
6. Remove duplicate characters from a string.
7. Keep only the first occurrence of each character.
8. Remove consecutive duplicate characters (e.g., "aaabb" → "ab").
9. Swap case: uppercase → lowercase and lowercase → uppercase.
10. Shift each character by 1 (e.g., "abc" → "bcd").

Category 5: Word-level Thinking

1. Print each word of a sentence on a new line.
2. Count how many words have even length.
3. Find the longest word in a sentence.
4. Find the shortest word in a sentence.
5. Swap first and last words in a sentence.
6. Print all words that start and end with the same letter.
7. Count how many words contain the letter 'a'.
8. Capitalize the first letter of each word.
9. Print the sentence in title case (first letter capital, rest lowercase).
10. Remove extra spaces between words (normalize spacing).

Phase 6 — Mixed Logical Challenges

Goal: Strengthen logical thinking with character manipulation.

Topics covered: char array logic, string length, substring, conditions.

Target Questions: 30–40

Category 1: Number-Based Logical Combinations

1. Print all numbers between 1 and N that are divisible by both 3 and 5.
2. Find the sum of digits of a number (use loop).
3. Check if a number is an Armstrong number.
4. Print all Armstrong numbers between 1 and 1000.
5. Find the factorial of a number using recursion.
6. Count how many even digits a number contains.
7. Print all prime numbers between 1 and N.
8. Print the reverse of a number (123 → 321).
9. Check if a number is palindrome (121 → true).
10. Check if a number is perfect (sum of factors equals number).

Category 2: String + Logic Mix

1. Check if two strings are anagrams (without using collections).
2. Count vowels in each word of a string.
3. Reverse words in a string if their length is even.
4. Replace every vowel in a string with its position (a=1, e=2...).
5. Print characters that appear more than once (without map).
6. Count words that start and end with the same letter.
7. Toggle case for every alternate word in a sentence.
8. Check if two strings are rotations of each other.
9. Find the word with maximum vowels in a sentence.
10. Remove duplicate words from a sentence.

Category 3: Array + Looping Logic

1. Find the maximum and minimum element in an array.
2. Count how many positive, negative, and zero elements are in an array.
3. Print all unique elements from an array.
4. Reverse an array in-place.

5. Shift all zeros to the end of the array.
6. Count how many elements are even at an even index.
7. Merge two arrays into one.
8. Find the second largest element in an array.
9. Rotate an array by one position to the right.
10. Find the sum of all elements at odd indices.

Category 4: Nested Logic & Pattern Flow

1. Print a multiplication table in a formatted grid (10x10).
2. Print all pairs in an array whose sum equals a given number.
3. Print all subarrays of a given array.
4. Check if an array is sorted (ascending or descending).
5. Count how many times a number appears consecutively in an array.
6. Find all pairs of characters in a string that are the same (nested loop).
7. Print pattern of increasing characters (A, AB, ABC...).
8. Print Pascal's triangle up to N rows.
9. Generate Fibonacci series up to N using recursion.
10. Print numbers in a spiral-like pattern (conceptual dry run).

Category 5: Applied Reasoning & Real-Life Logic

1. Given marks of students, find how many passed (>= 40).
2. Take age inputs and count how many are adults, minors, seniors.
3. Validate a password (at least one uppercase, lowercase, digit, special char).
4. Simulate a simple calculator using switch-case.
5. Count how many times a coin lands on heads/tails (use random).
6. Print frequency of each digit in a number.
7. Find common elements between two arrays.
8. Print characters that are common in two strings.
9. Count how many prime numbers are there in an array.
10. Print all palindromic words from a sentence.

Phases 7–10 — C-Specific Prerequisites (New, Fills the DSA Gap)

Phase 7 — Functions (Modular Thinking)

Goal: Learn to decompose logic into reusable functions — every DSA operation (insert, delete, search, traverse) will be written as one.

Topics covered: function declaration/definition, parameters, return values, call by value, scope, recursion recap via functions, function prototypes, void functions.

Target Questions: 30–40

Level 1: Fundamentals

1. Write a function to add two numbers and return the result.
2. Write a function to check if a number is even or odd (return 1/0).
3. Write a void function that prints a welcome message n times.
4. Write a function to find the factorial of a number.

5. Write a function to check if a number is prime.
6. Write a function that takes no parameters and returns a random-looking constant value.
7. Write a function to swap two numbers (observe why call-by-value fails here).
8. Write a function to find the maximum of three numbers.
9. Write a function to convert Celsius to Fahrenheit.
10. Write a function prototype, definition, and call in three separate blocks to understand structure.

Level 2: Parameters & Return Logic

1. Write a function that takes an array and its size, and returns the sum.
2. Write a function that takes an array and returns the maximum element.
3. Write a function that takes a string and returns its length (without strlen).
4. Write a function isPalindrome(char str[]) that returns 1 or 0.
5. Write a function power(int base, int exp) using recursion.
6. Write a function gcd(int a, int b) using recursion.
7. Write a function that takes an array and reverses it in place (void, modifies the caller's array).
8. Write a function that takes marks and returns a grade character.
9. Write an overloading-style set of functions in C using different names for int and float area calculations (since C has no true overloading) — areaCircleInt, areaCircleFloat.
10. Write a function that returns multiple pieces of information conceptually — return an int but also modify a variable via a pointer parameter (this bridges into Phase 8).

Level 3: Scope, Recursion & Applied Function Design

1. Demonstrate the difference between a local variable and a global variable using two functions.
2. Write a static local variable inside a function that counts how many times the function has been called.
3. Write a menu-driven program using functions for each operation (add/subtract/multiply/divide) via switch-case.
4. Write a function to check balanced parentheses in a string using a simple counter (precursor to stack logic).
5. Write a function that computes nCr using two helper functions (factorial as a sub-function).
6. Write a recursive function to compute the sum of an array using function calls only (no loop).
7. Write a function linearSearch(int arr[], int n, int key) returning the index or -1.
8. Write a function binarySearch(int arr[], int n, int key) on a sorted array.
9. Write a function bubbleSortStep(int arr[], int n) that performs one pass of bubble sort — call it n times from main.
10. Design a small library of 3–4 functions (init, insert, display, sum) that operate on the same global array — a direct rehearsal for how a linked list or stack API will be structured in C.

Why this matters: Every DSA operation in C — push(), pop(), insertNode(), search() — is a function. If function design and call-by-value/reference isn't second nature, DSA code becomes copy-pasted rather than understood.

Phase 8 — Pointers (Address & Memory Logic)

Goal: Understand memory addresses, dereferencing, and pointer arithmetic — the single most important C prerequisite for DSA.

Topics covered: address-of (&) and dereference (*) operators, pointer declaration, pointer arithmetic, pointers with arrays, pointers with functions (call by reference), pointer to pointer, NULL pointers, void pointers.

Target Questions: 40–50

Level 1: Pointer Fundamentals

1. Declare an integer variable and print its address using &.
2. Declare a pointer to an int, assign it the address of a variable, and print the value via dereferencing.
3. Change the value of a variable indirectly through its pointer.

4. Declare a pointer to a float and a pointer to a char; observe pointer size vs data size.
5. Print the address stored in a pointer and the address of the pointer itself.
6. Demonstrate a NULL pointer and check for it before dereferencing (if (ptr != NULL)).
7. Take two integers and swap them using a function with pointer parameters (call by reference) — compare with the Phase 7 call-by-value attempt.
8. Write a function to add 1 to a variable using a pointer parameter.
9. Declare a pointer to a pointer (int **pp) and access the original value through two levels of dereference.
10. Demonstrate that an uninitialized pointer (wild pointer) is dangerous — explain conceptually why, without dereferencing it.

Level 2: Pointers with Arrays

1. Declare an array and a pointer to its first element; print elements using pointer arithmetic (*(ptr+i)) instead of arr[i].
2. Traverse an array using only a pointer and pointer increment (ptr++) in a loop.
3. Find the sum of an array using pointer arithmetic only.
4. Find the maximum element in an array using a pointer-based traversal.
5. Reverse an array in place using two pointers moving toward each other (this is the direct two-pointer technique used constantly in DSA).
6. Write a function that accepts an array as a pointer parameter (int *arr) and computes the average.
7. Compare the addresses of consecutive array elements to confirm contiguous memory layout.
8. Copy one array into another using pointer traversal (no array indexing).
9. Find the first and last occurrence of an element using pointer traversal.
10. Implement a simple string length function using a char pointer and pointer arithmetic (mirrors how strlen works internally).

Level 3: Pointers with Functions & Strings

1. Write a function that returns a pointer to the maximum element in an array.
2. Write a function that takes a char pointer and reverses the string in place using pointer swaps.
3. Write a function to check if a string is a palindrome using two pointers (start and end).
4. Write a function to concatenate two strings manually using pointers (no strcat).
5. Write a function to copy a string manually using pointers (no strcpy).
6. Write a function to compare two strings manually using pointers (no strcmp).
7. Write a function that takes an array of pointers to strings (char *names[]) and prints them all.
8. Demonstrate the difference between array of pointers and pointer to array with a small example.
9. Write a function using a void pointer parameter and cast it to int* and float* in two different calls.
10. Write a function that dynamically decides which of two integers is larger, returning a pointer to the larger variable.

Level 4: Applied Pointer Problems (Direct DSA Rehearsal)

1. Simulate a single 'node' using two variables (int data, and an address-holder) and manually link two of them together — the exact idea behind a linked list node, done without struct first.
2. Write a function that takes a pointer to an integer and increments the value it points to, called from a loop to simulate a running counter across function calls.
3. Implement the two-pointer technique to find if an array has a pair that sums to a target value.
4. Implement a fast and slow pointer walk over an array (moving one pointer twice as fast) — this is the exact technique used later to detect cycles in linked lists.
5. Write a function that takes a pointer to an array and its length, and shifts all elements one position using pointer arithmetic only.
6. Simulate a simple 'swap by reference' utility function and use it inside a basic bubble sort implementation.

7. Write a function that returns a dynamically-sized array's pointer (conceptually — return the pointer from a function, understanding it must outlive the function's stack frame).
8. Use a pointer to traverse a 2D array (int arr[3][3]) row by row using pointer arithmetic instead of double indexing.
9. Write a function that takes a pointer to a pointer to swap which array a caller is 'looking at' (int **selector).
10. Trace through, on paper or in comments, what happens in memory when a pointer is passed through three nested function calls — write this as commented pseudocode inside a working C program.

Why this matters: A linked list node is just a struct with a pointer to the next node. A tree node has two or more such pointers. If pointer arithmetic and call-by-reference aren't fluent, every single linked-list/tree/graph operation in DSA will feel like magic instead of logic.

Phase 9 — Structures & User-Defined Data Types

Goal: Learn to group related data into a single unit — this is what a 'Node' actually is in C.

Topics covered: struct declaration, accessing members (. and ->), struct with pointers, typedef, nested structures, arrays of structures, structures with functions.

Target Questions: 25–30

Level 1: Struct Fundamentals

1. Define a struct Student with name, roll number, and marks; declare a variable and print its members.
2. Define a struct Point with x and y; write a function to compute the distance between two Points.
3. Use typedef to simplify a struct declaration (typedef struct {...} Point;).
4. Define a struct Rectangle with length and width; write a function to compute its area.
5. Create an array of 5 struct Student and fill it using a loop; print all records.
6. Define a struct Date (day, month, year) and write a function to check if it's a leap year.
7. Write a function that takes a struct Point by value and returns a new, shifted Point.
8. Define a struct Employee with a nested struct Date (joining date) inside it.
9. Compare two struct Point values for equality by comparing their members.
10. Write a function to swap two struct Student records (by value, whole-struct swap).

Level 2: Structures with Pointers (Direct Node Rehearsal)

1. Declare a pointer to a struct Student and access its members using the arrow operator (->).
2. Write a function that takes a struct Point* and modifies its coordinates directly.
3. Define a struct Node with an int data field and a struct Node* next field — this IS a linked list node; do not link anything yet, just declare it correctly.
4. Create two struct Node variables manually and set the first node's next pointer to the address of the second — a two-node manual chain, traversed and printed with a loop.
5. Write a function createNode(int value) that allocates a struct Node on the stack (not yet with malloc) and returns it by value.
6. Write a function to print all fields of a struct Employee using a pointer parameter.
7. Build a struct Rectangle array, then write a function to find which rectangle has the largest area using pointers to iterate.
8. Define a struct Stack (very small, fixed array + top index) and write push/pop functions that operate on a struct Stack* parameter.
9. Define a struct Fraction (numerator, denominator) with an add function that returns a new struct Fraction.
10. Write a compareStudents(struct Student *a, struct Student *b) function returning who has higher marks, used to conceptually sort an array of structs.

Why this matters: struct Node { int data; struct Node* next; }; is the literal definition of a linked list node. Every tree, graph, stack, and queue implementation in C starts with a struct exactly like the ones above.

Phase 10 — Dynamic Memory Allocation

Goal: Learn to create and destroy memory at runtime — required for every non-fixed-size DSA structure.

Topics covered: malloc, calloc, realloc, free, dynamic arrays, memory leaks, dangling pointers, sizeof with dynamic allocation.

Target Questions: 20–25

Level 1: Fundamentals

1. Dynamically allocate a single integer using malloc, assign it a value, print it, then free it.
2. Dynamically allocate an array of n integers (size taken from user input) using malloc.
3. Repeat the above using calloc and observe/explain the difference (zero-initialization).
4. Write a program that allocates memory, forgets to free it, and explain (in a comment) why this is a memory leak.
5. Allocate a dynamic array, fill it with user input, print the sum, then properly free it.
6. Demonstrate a dangling pointer: free a pointer, then explain (in comments, not by dereferencing) why using it afterward is undefined behavior.
7. Dynamically allocate memory for a struct Point using malloc and access its members using ->.
8. Compare sizeof(int) * n against the size passed to malloc to confirm correct allocation size.
9. Check malloc's return value for NULL before using the allocated memory (out-of-memory handling).
10. Allocate and free memory for 3 different data types (int, float, struct) in the same program to build the habit.

Level 2: Dynamic Arrays & Resizing

1. Build a dynamic array that starts at capacity 2 and doubles its capacity using realloc whenever it's full — a minimal version of what a dynamic array / ArrayList does internally.
2. Write an insertAt(int *arr, int index, int value) style function operating on dynamically allocated memory.
3. Implement a growable array where the user keeps entering numbers until they type -1, resizing with realloc as needed.
4. Write a function that takes an existing dynamic array, its current size, and returns a new, larger dynamically allocated array with the old data copied over (manual realloc logic).
5. Free a 2D dynamically allocated array (array of pointers, each pointing to a malloc'd row) correctly, row by row, then the row-pointer array itself.

Level 3: Applied — Building Your First Real Node

1. Combine Phase 9's struct Node with malloc: write createNode(int value) that uses malloc to allocate a new node on the heap, sets data and next = NULL, and returns the pointer — this is the actual function you will reuse in every linked-list program from here on.
2. Create two nodes using createNode() and manually link them (first->next = second); traverse and print the 2-node chain, then free both nodes.
3. Write a function to create n nodes in a loop, linking each to the previous one, forming a simple chain (a linked list, informally, without naming it yet).
4. Traverse the chain built above and free every node one at a time without losing the reference to the next node (the correct free-while-traversing pattern used to avoid memory leaks in real linked-list deletion).
5. Write a function countNodes(struct Node *head) that walks the chain and returns its length — your first real linked-list-style function.

Why this matters: malloc + struct + pointer is the exact combination behind every 'insert a node' operation in linked lists, trees, and graphs. Phase 10, Level 3 is quite literally your first linked list, built with full understanding instead of copied syntax.

Readiness Checklist Before Starting DSA in C

1. Comfortable writing and calling functions with parameters and return values.

2. Comfortable using & and * without hesitation, and explaining call-by-value vs call-by-reference.
3. Comfortable declaring a struct with a self-referential pointer (struct Node* next) and explaining what it means.
4. Comfortable allocating memory with malloc, checking for NULL, and freeing it — without leaks or dangling pointers.
5. Have manually built and freed a small linked chain of nodes (Phase 10, Level 3) at least once.

Once all five are true, you are genuinely ready — not just theoretically, but practically — to start Linked Lists, Stacks, Queues, Trees, and Graphs in C.